Project Report
Distorted Visual Sequence Pattern Recognition

Submitted by

Rishav Kumar

Enrollment No: 24323032

1. Dataset Analysis and Exploratory Data Analysis

Before writing a single line of model code, the first step was to thoroughly understand what the data actually looked like.

1.1 Dataset Overview

The training set consists of 20,000 grayscale images paired with ground-truth text labels stored in a CSV file (train-labels.csv). Each image is named sequentially (train-0.png, train-1.png, ...) and the test images follow the same convention (test-0.png, test-1.png, ...).

Property	Value
Total Training Samples	20,000
Image Format	Grayscale PNG
Image Size (resized)	128 x 32 pixels
Sequence Length Range	6 to 9 characters (strict)
Vocabulary Size	38 unique characters
Vocabulary	+-.0123456789ABCDEFGHJKMNPQRSTUVWXYZar

1.2 Visual Challenges

- Heavy pepper noise spread across the image surface
- Font overlapping, where characters appeared to blend or bleed into each other
- Massive black occlusion blobs, the most severe challenge, which completely covered chunks of the underlying text

1.3 Augmentation Strategy

Augmentations were handled using the Albumentations library. The core design principle was to focus only on spatial invariance and avoid adding any additional occlusion or dropout on top of what the dataset already provides. Adding more blackout regions on images that are already heavily occluded would make characters mathematically invisible during training and destroy the learning signal entirely.

The final augmentation pipeline applied to the training set was:

- ShiftScaleRotate: slight shifts (5%), scale jitter (5%), and rotations up to 4 degrees with replicated border padding, applied with 50% probability
- GridDistortion: mild grid-based warping with 3 steps and a distortion limit of 0.1, applied with 30% probability

No augmentation was applied at validation or test time. The train/val split was 90/10 (18,000 train, 2,000 val) with a fixed random seed of 42 for reproducibility.

2. Approach 1: Naive CNN Baseline (CTC Loss)

2.1 Motivation

Before building anything complex, the goal here was purely to verify that the data pipeline, custom collate functions, and tensor shapes were all mathematically correct. The architecture itself was intentionally kept as simple as possible.

2.2 Architecture

The model is a plain Convolutional Neural Network with four convolutional blocks feeding into two dense layers, with a final log-softmax output for CTC loss.

- 4 convolutional blocks with BatchNorm and ReLU, progressively expanding channels: 32, 64, 128, 256
- MaxPool2d reductions bring the feature map from (1, 32, 128) down to (256, 2, 32)
- The spatial dimensions are merged into a feature vector of size 512, forming a sequence of 32 time steps
- Two dense layers (512 -> 256 -> num_classes) with dropout of 0.2
- Loss function: nn.CTCLoss with blank token at index 0

2.3 Training Setup

Optimizer: Adam with a learning rate of 0.001. Scheduler: ReduceLROnPlateau (factor 0.5, patience 3). Batch size: 64. Epochs: 10.

2.4 Results

Metric	Value
Total Parameters	530,151
Best Validation CER	0.9351 (6.49% character accuracy)
Exact Sequence Match	0.00%
Throughput	~8,882 FPS (0.11 ms per image)

2.5 Analysis and Failure Modes

A pure CNN processes each column of the image independently and has no memory of what it has seen before. When a column of pixels is fully covered by a black occlusion blob, the network has zero context to draw on. It cannot look left or right at neighbouring characters to guess what might be hidden.

The CTC loss mechanism also requires some detectable signal at each time step to align properly. With large regions producing nothing but black pixels, the loss function cannot anchor its alignment and collapses into random or blank predictions. The 0.00% exact match rate confirms the model never once correctly decoded a full sequence.

Hence, this approach failed.

3. Approach 2: CRNN + Spatial Transformer Network (CTC Loss)

3.1 Motivation

The CRNN architecture (Convolutional Recurrent Neural Network) is the standard production approach for OCR tasks and was expected to perform significantly better. A Spatial Transformer Network (STN) was added at the front to attempt to mathematically correct the spatial distortions before the backbone processed the image.

3.2 Architecture

The architecture consists of three major components stacked in sequence.

Spatial Transformer Network (STN)

A small localization network (two conv layers with pooling) predicts the 6 parameters of a 2x3 affine transformation matrix. The network is initialized to the identity transform so it starts with no warp and learns corrections gradually. The affine_grid and grid_sample functions apply the predicted warp to the image before it reaches the backbone.

VGG-Style CNN Backbone

A deeper backbone than Approach 1, with additional convolutional blocks. The pooling strategy is adjusted so the height is fully collapsed to 1 while the width (time dimension) is preserved, producing a sequence of feature vectors.

Bidirectional LSTMs

Two stacked Bidirectional LSTM layers (256 hidden units each) process the CNN feature sequence. This gives the model explicit left-to-right and right-to-left context at each time step. Output is projected to the CTC vocabulary size and trained with CTCLoss.

3.3 Training

The model was trained for 120 epochs with Adam optimizer, scheduled learning rate reductions, and gradient clipping at 5.0.

3.4 Results and Failure Analysis

The model showed almost no learning for the first 28 epochs, with Validation CER hovering between 0.94 and 0.97. It briefly converged to a CER of 0.9055 at epoch 30 before progress stalled again.

The failure was caused by CTC Blank Collapse. Because the CRNN processes images strictly left-to-right, large black occlusions create gaps in the usable signal, causing the model to lose context and reset its hidden state. Since CTC loss enforces strict alignment, the model learned to simply predict blank tokens for occluded areas to minimize loss. The Spatial Transformer Network (STN) could not assist, as it only corrects global warping and cannot reconstruct data completely destroyed by occlusions.

4. Approach 3: 1D Transformer OCR (Seq2Seq with Cross-Attention)

4.1 Motivation and Key Design Shift

The attention mechanism can look at the entire encoded image sequence at once, from any decoding step.

4.2 Architecture

CNN Encoder

A four-block CNN extracts visual features from the input image. The pooling strategy partially collapses the height while preserving the width, producing a sequence of 32 spatial feature tokens of dimension 256. These tokens are the memory the decoder attends to.

Transformer Decoder

Three TransformerDecoderLayer blocks (PyTorch standard), each with 8 attention heads, a feedforward dimension of 512, and dropout of 0.2. The decoder uses a standard causal (look-ahead) mask during training so each output position can only attend to previously generated tokens. At inference time, tokens are generated autoregressively.

Positional Encoding

A sinusoidal PositionalEncoding module is applied to the embedded target sequence before it enters the decoder. This provides position information that the attention mechanism itself does not inherently possess.

Output Head and Loss

A single linear layer projects from the 256-dimensional decoder output to the 41-class vocabulary (38 characters plus PAD, SOS, EOS tokens). Loss is standard nn.CrossEntropyLoss ignoring PAD tokens. This replaces CTC entirely.

4.3 Training Setup

Optimizer: AdamW with learning rate 5e-4 and weight decay 1e-4. Scheduler: ReduceLROnPlateau (factor 0.5, patience 4). Batch size: 64. Epochs: 40. Gradient clipping: 1.0.

The seq2seq collate function pads each target sequence with SOS at the start and EOS at the end, padding shorter sequences to the maximum length in the batch with PAD tokens. Teacher forcing is used during training: the decoder receives the ground-truth tokens as input at each step.

4.4 Results

Metric	Value
Total Parameters	3,354,153
Epochs Trained	40
Best Validation CER	0.0050 (99.50% character accuracy)
Exact Sequence Match	97.55%
Throughput	~433 FPS (2.31 ms per image)

4.5 Why It Worked

This was the major breakthrough of the project. The Cross-Attention mechanism in the Transformer decoder does not read the image sequence from left to right. At each decoding step, it computes attention weights over all 32 encoder tokens simultaneously. When a character is hidden under a black blob, the attention for that decoding step simply shifts weight to the visible tokens on either side of the blob, using the surrounding characters and partial stroke information to infer the missing letter.

There is no temporal state that gets wiped out by the occlusion. The decoder has access to the full encoded representation at every step, and the attention mechanism dynamically selects which parts of that representation are most useful. The model effectively learned to route attention around the occlusion blobs rather than through them.

5. Approach 4: 2D Spatial Attention Transformer

5.1 Motivation

The 2D Transformer improves accuracy and speed over the 1D model by preserving vertical spatial features through a 4x16 grid, without increasing parameter count.

5.2 Architectural Change

The only modification to the 1D Transformer was CNN's final pooling strategy. Instead of collapsing height down to 1 (producing a 1x32 strip of tokens), the pooling layers are adjusted to produce a 4x16 spatial grid. This grid is then flattened into 64 spatial tokens rather than 32.

The decoder architecture, embedding dimension, number of heads, and number of layers are all identical to Approach 3. The parameter count remains exactly 3,354,153.

5.3 Benchmark Results (12 Epoch Comparison)

Metric	1D Transformer (Epoch 12)	2D Transformer (Epoch 12)
Validation CER	0.0267	0.0232
Character Accuracy	97.33%	97.68%
Throughput	433 FPS	527 FPS (+21.6%)
Latency	2.31 ms / image	1.90 ms / image
Parameters	3,354,153	3,354,153 (identical)

5.4 Why It Improved Both Accuracy and Speed

Two factors explain the improvement.

On accuracy: by preserving the 2D layout of visual features, the attention heads have a more natural coordinate system for reasoning about character strokes. They can attend to the region above or below an occlusion blob in a geometrically meaningful way, rather than treating all 32 tokens as an undifferentiated 1D sequence.

On speed: the 4x16 grid layout, while producing more tokens (64 vs 32), maps more efficiently to GPU memory access patterns. The flatter, more square spatial layout results in better cache utilization compared to the very elongated 1x32 strip. This is purely a hardware efficiency gain from the spatial layout change, and it translated to a 21.6% throughput improvement, dropping latency below 2 milliseconds per image.

6. Final Summary and Submission

6.1 Architecture Progression Overview

Approach	Architecture	Loss	Val CER	Exact Match	FPS
1	Naive CNN	CTC	0.9351	0.00%	8,882
2	CRNN + STN	CTC	~0.90 (stalled)	N/A	N/A
3	1D Transformer Seq2Seq	CrossEntropy	0.0050	97.55%	433
4	2D Spatial Transformer	CrossEntropy	0.0232 (ep 12)	N/A	527

6.2 Key Findings

The project demonstrated that autoregressive Cross-Entropy loss with global attention significantly outperforms CTC loss for heavily occluded text, as CTC's strict temporal alignment fails when signal is missing. Furthermore, reorganizing CNN features into a 2D spatial grid improved both accuracy and hardware efficiency, proving that feature layout is as critical as model architecture.

6.3 Submitted Model

The final submission file (submission_rishav_24323032.csv) was generated using the 1D Transformer checkpoint (best_seq2seq_ocr.pth) saved at the best validation CER across 40 epochs. The 1D Transformer was selected for submission over the 2D variant because it was trained to full convergence (40 epochs vs the 12-epoch benchmark run), giving it the advantage on final test set accuracy.

The inference pipeline loads the checkpoint, runs autoregressive greedy decoding with a maximum length of 12 tokens, and writes predictions to CSV. The entire test set is processed in batch using DataLoader with a batch size of 128 for efficiency.

6.4 Reproduction Instructions

To reproduce the submission file:

- Place best_seq2seq_ocr.pth and the test_images/ directory in the same working directory
- Run the final cell of distorted_ocr_submission.ipynb
- The output file submission_rishav_24323032.csv will be generated in the same directory

All random seeds are fixed (torch.Generator().manual_seed(42) for the train/val split) to ensure reproducibility of the evaluation setup.

Appendix: Model Parameter Counts and Configuration Details

A. Vocabulary and Token Mapping

Token Type	Index	Description
PAD	0	Padding token (ignored by CrossEntropyLoss)
SOS	1	Start-of-sequence token
EOS	2	End-of-sequence token
Characters	3 to 40	38 vocab characters in order
CTC Blank	0	Blank token for CTC loss (separate mapping)

B. Hyperparameter Summary

Parameter	Approach 1 & 2 (CTC)	Approach 3 & 4 (Seq2Seq)
Optimizer	Adam	AdamW
Initial LR	0.001	0.0005
Weight Decay	None	0.0001
Batch Size	64	64
Gradient Clip	5.0	1.0
Scheduler	ReduceLROnPlateau	ReduceLROnPlateau
LR Factor	0.5	0.5
LR Patience	3 epochs	4 epochs